

SWEN90010 Predicted Mock Exam - 14 Questions, 60 Marks

Source basis:

- materials/2023-pastpaper.pdf
- materials/2024-pastpaper.pdf
- materials/Sample_Exam.pdf
- materials/learning_outcomes.pdf
- Local lecture notes in outputs/notes/

This is a predicted practice paper, not an official exam paper. It is designed to match the observed exam style: scenario-based safety/security questions, Alloy modelling and assertions, Ada/SPARK contract reasoning, Hoare logic, and pointer/separation-logic reasoning.

Visible total: 60 marks across 14 questions.

Suggested practice rule: answer one question at a time. After you answer, fill the corresponding Full-Mark Answer section during feedback.

Q1. HAZOP Mini-Analysis - 4 marks

A hospital uses an automated medication trolley. A nurse scans a patient wristband and a medicine barcode. The trolley asks the hospital prescription service whether the medicine and dose are valid for that patient. If the answer is valid, the trolley unlocks the correct medicine drawer.

Perform a small HAZOP analysis on the process item:

valid dispense authorisation is sent to the drawer-lock controller

Choose two suitable HAZOP guideword deviations from the list below and complete the table.

Guidewords:

- NO
- MORE
- LESS
- AS WELL AS
- OTHER THAN
- EARLY
- LATE

Guideword and deviation	Possible consequence	Possible cause	Safeguard or mitigation

Q1 Full-Mark Answer

Guideword and deviation	Possible consequence	Possible cause	Safeguard or mitigation
NO: no valid dispense authorisation is sent to the drawer-lock controller	Correct medicine drawer stays locked; treatment is delayed, or nurse may use an unsafe manual workaround	Network/API failure, prescription service unavailable, barcode scan not accepted, controller message lost	Fail closed, show a clear alarm, allow controlled pharmacist/nurse override, log the failed authorisation
OTHER THAN: authorisation is for the wrong patient, medicine, dose, or drawer	Wrong drawer unlocks; patient may receive the wrong medicine or wrong dose	Barcode mix-up, stale prescription data, software mapping bug, tampered authorisation message	Re-check patient ID, medicine ID, dose, and drawer ID at the controller; signed messages; nurse confirmation; audit log

Memorise: HAZOP answer = guideword + deviation -> harm -> cause -> control.

Reason: The guideword must modify the process item. Do not write only "NO" or "OTHER THAN"; write what is absent or wrong. A high-mark answer connects the deviation to patient harm and then gives a practical safeguard.

Q2. Fault-Tree Analysis - 5 marks

A warehouse robot has an automatic stopping system. The robot should stop when a human is detected in its path.

The system has:

- a lidar sensor,
- a camera sensor,
- a controller, and
- a brake actuator.

The robot detects a human if either the lidar sensor or the camera sensor detects a human. The controller sends a brake command if a human is detected and the controller has not failed. The brake actuator stops the robot if it receives a brake command and the actuator has not failed.

The hazard is:

human in robot path and robot does not stop

The basic failure events are:

- lidar sensor fails,
- camera sensor fails,
- controller fails,
- brake actuator fails.

Write a fault tree for this hazard using preformatted text. Use + OR and + AND gates as needed.

Q2 Full-Mark Answer

```
human in robot path and robot does not stop
+ AND
  human is in robot path
  robot does not stop
  + OR
    brake actuator fails
    brake command is not sent
  + OR
    controller fails
    human is not detected
  + AND
    lidar sensor fails
    camera sensor fails
```

Memorise: Work backwards from the hazard. robot does not stop happens if the actuator fails OR no brake command arrives. No brake command happens if the controller fails OR detection fails. Detection fails only if both redundant sensors fail.

Reason: The lidar and camera are in an OR detection structure: either one can detect the human. Therefore missed detection requires an AND of both sensor failures. The actuator and controller are single points of failure, so either one can independently cause the stopping failure.

Q3. Fault-Tolerant Design Choice - 3 marks

Continuing the warehouse robot scenario from Q2.

The manufacturer observes that the robot sometimes stops unnecessarily because one sensor falsely reports a human. A manager suggests changing the system so that the robot stops only when both the lidar and the camera detect a human.

Is this a good idea for a safety-critical stopping system? Explain briefly. If it is not a good idea, suggest a better fault-tolerant design.

Your answer should be no more than about 5 sentences.

Your answer:

Q3 Full-Mark Answer

No. Requiring both sensors to detect a human is unsafe for a safety-critical stopping system because one missed detection would be enough to prevent stopping. It reduces false stops, but it increases the risk of a false negative, which is usually worse in a human-in-path hazard. A better design is to keep a fail-safe stop-on-uncertainty rule, or use three independent/diverse sensors with majority voting and diagnostics so that one faulty sensor can be masked. The system should prefer a safe unnecessary stop over failing to stop for a real human.

Memorise: 2 sensors with AND hides false alarms but makes one missed detection dangerous. 3 sensors with voting can mask one faulty sensor.

Reason: For safety, the dangerous failure is "human present but robot continues." Changing from lidar OR camera to lidar AND camera makes that dangerous failure easier to reach.

Q4. Trust Boundaries and STRIDE Properties - 4 marks

Consider a medication ordering system with the following components:

- patients using a mobile app,
- nurses using a hospital workstation,
- a hospital web API,
- a hospital medication database,
- a hospital identity service, and
- a third-party payment provider.

The hospital controls the web API, medication database, identity service, and workstation environment. It does not control patient phones or the third-party payment provider.

1. [2 marks] Identify the trust regions in this system and state how many trust boundaries are relevant for threat modelling.
2. [2 marks] Match each STRIDE category to the security property it threatens.

STRIDE category	Security property
Spoofing	
Tampering	
Repudiation	
Information disclosure	
Denial of service	
Elevation of privilege	

Q4 Full-Mark Answer

1. The main trust regions are:

- Hospital-controlled region: hospital web API, medication database, identity service, and hospital workstation environment.
- Patient-device region: patient mobile phones and mobile app environment.
- Third-party region: payment provider.

There are two main trust boundaries for this scenario: patient phone/app to hospital API, and third-party payment provider to hospital API.

2. STRIDE mapping:

STRIDE category	Security property
Spoofing	Authentication
Tampering	Integrity
Repudiation	Non-repudiation / accountability
Information disclosure	Confidentiality
Denial of service	Availability
Elevation of privilege	Authorisation / access control

Memorise: STRIDE properties = S-authentication, T-integrity, R-non-repudiation, I-confidentiality, D-availability, E-authorisation.

Reason: Trust boundaries appear where control changes. The hospital controls its internal systems, but it does not control patient phones or the third-party payment provider, so messages crossing those edges are threat-modelling targets.

Q5. STRIDE Threat Generation - 6 marks

Continuing the medication ordering system from Q4.

For each STRIDE category, give one concrete threat that is specific to this architecture. A good answer names the attacker or faulty component, the affected component/message/data, and the security property being broken.

STRIDE category	Concrete threat
Spoofing	
Tampering	
Repudiation	
Information disclosure	
Denial of service	
Elevation of privilege	

Q5 Full-Mark Answer

STRIDE category	Concrete threat
Spoofing	An attacker steals a patient's session token from a phone and calls the hospital API as that patient.
Tampering	A compromised phone or proxy modifies a medication-order request before it reaches the hospital API.
Repudiation	A nurse or patient denies submitting an order because the system lacks signed requests or reliable audit logs.
Information disclosure	A compromised phone, API endpoint, or payment integration leaks prescription or patient-identifying information.
Denial of service	An attacker floods the hospital API or payment callback endpoint so legitimate medication orders cannot be processed.
Elevation of privilege	A patient exploits a missing access-control check to use a nurse-only API operation or view another patient's medication order.

Memorise: Good STRIDE threat sentence = attacker/component + action + target + broken property.

Reason: A generic phrase like "someone attacks the server" is weak. Full marks come from making the threat architecture-specific: phone, API, identity service, database, workstation, or payment provider.

Q6. Alloy: State Model, Function Constraint, and Permissions - 5 marks

The following Alloy model describes part of a medical-record system. A record has some contents and an access-control list. The state maps record names to records.

```
sig Data {}
sig User {}
sig RecordName {}

abstract sig Permission {}
```

```

one sig Read, Write extends Permission {}

sig Record {
  contents : Data,
  acl : User -> Permission
}

one sig State {
  var records : RecordName -> Record
}

```

1. [2 marks] Write a fact stating that, in every state, each record name maps to at most one record.

2. [3 marks] Write an Alloy function:

```

fun permissions[u : User, n : RecordName, s : State] : set Permission {
  // FILL IN HERE
}

```

Work area:

Q6 Full-Mark Answer

```

fact record_names_are_functional {
  always all n : RecordName | lone State.records[n]
}

fun permissions[u : User, n : RecordName, s : State] : set Permission {
  s.records[n].acl[u]
}

```

Memorise: Functional mapping constraint = `always all key | lone relation[key]`. Permission lookup = `name -> record -> acl -> user`.

Reason: `RecordName -> Record` allows one name to map to many records unless a fact rules that out. `s.records[n]` gets the record for name `n`; `.acl[u]` then gets the permissions assigned to user `u`. If the record does not exist, the expression returns the empty set.

Q7. Alloy: Assertion Interpretation and Critique - 4 marks

Continuing the medical-record model from Q6.

Suppose the system has an operation `updateRecord[u, n, d, s]` that updates record `n` to contain data `d` on behalf of user `u` in pre-state `s`.

We define:

```

pred state_transition[u : User, s : State] {
  some n : RecordName, d : Data | updateRecord[u, n, d, s]
}

```

Consider the following assertion:

```

assert write_safe {
  all u : User, s : State | state_transition[u, s] implies
    all n : RecordName |
      Write not in permissions[u, n, s] implies
        s.records'[n] = s.records[n]
}

```

1. [2 marks] Explain in words what this assertion is trying to check.

2. [2 marks] If `updateRecord` is not allowed to create new records, should a correct implementation satisfy this assertion? Explain why.

Your answer:

Q7 Full-Mark Answer

The assertion checks that a transition performed by user `u` cannot change any record name `n` unless `u` had `Write` permission for `n` in the pre-state `s`. In other words, if `Write not in permissions[u, n, s]`, then the mapping for `n` must be the same

after the transition as before it.

Yes, if `updateRecord` is not allowed to create new records, a correct implementation should satisfy this assertion. Existing records can only be updated when the user has `Write`, and non-existing names must remain non-existing because creation is forbidden. Therefore every name without pre-state `Write` permission should be unchanged.

Memorise: Assertion meaning = no `Write` before -> same mapping after.

Reason: The key phrase is "pre-state permission". The assertion does not ask whether the user gets `Write` later; it protects records according to the permission held before the transition.

Q8. Alloy: Operation Predicate - 6 marks

Continuing the medical-record model from Q6.

Complete the predicate `updateRecord`. It should model the following operation:

- The record name `n` must already exist.
- User `u` must have `Write` permission to record `n` in pre-state `s`.
- The post-state record at name `n` has contents `d`.
- The record's ACL is unchanged.
- No other record-name mapping changes.

```
pred updateRecord[u : User, n : RecordName, d : Data, s : State] {  
  // FILL IN HERE  
}
```

Work area:

Q8 Full-Mark Answer

```
pred updateRecord[u : User, n : RecordName, d : Data, s : State] {  
  some s.records[n]  
  Write in permissions[u, n, s]  
  
  some newRecord : Record | {  
    newRecord.contents = d  
    newRecord.acl = s.records[n].acl  
    s.records' = s.records ++ (n -> newRecord)  
  }  
}
```

Memorise: Operation predicate = preconditions + post update + frame condition. For a map update, use override: `oldMap ++ (key -> newValue)`.

Reason: `some s.records[n]` says the record already exists. `Write in permissions[...]` is the authorisation check. Because `contents` and `acl` are fields of `Record`, the operation models an update by making `n` point to a record whose contents are `d` and whose ACL equals the old ACL. The override expression changes only the mapping for `n`; all other names keep their old mappings.

Q9. Ada Translation and Runtime Error - 3 marks

Consider the following C function:

```
int dec(int x) {  
  return x - 1;  
}
```

1. [2 marks] Translate this function directly into Ada using the standard `Integer` type.
2. [1 mark] Suppose Ada `Integer` is a 32-bit signed integer and the function is called with `X = Integer'First`. What runtime error can occur, and why?

```
-- Your Ada function
```

Runtime-error explanation:

Q9 Full-Mark Answer

```
function Dec (X : Integer) return Integer is
begin
    return X - 1;
end Dec;
```

If $X = \text{Integer}'\text{First}$, evaluating $X - 1$ goes below the lower bound of `Integer`. Ada checks integer range constraints at runtime, so this can raise `Constraint_Error`.

Memorise: `Integer'First - 1` underflows; `Integer'Last + 1` overflows.

Reason: Ada is safer than C here because the invalid arithmetic is detected instead of silently wrapping or causing undefined behaviour.

Q10. SPARK Weakest Error-Free Precondition - 4 marks

Consider the following Ada function:

```
function Subtract (A : Integer; B : Integer) return Integer is
begin
    return A - B;
end Subtract;
```

Write a suitable SPARK precondition that is no stronger than necessary to guarantee that `Subtract` is free from arithmetic overflow or underflow. Your answer must not evaluate $A - B$ directly in the precondition in a way that could itself overflow.

You may use mathematical integer notation, `Big_Integer`, or a safe case split over `B`.

```
-- Your precondition
```

Briefly explain why your precondition is the right strength.

Explanation:

Q10 Full-Mark Answer

Using mathematical integers:

```
with Pre =>
    To_Big_Integer (A) - To_Big_Integer (B) >= To_Big_Integer (Integer'First)
and
    To_Big_Integer (A) - To_Big_Integer (B) <= To_Big_Integer (Integer'Last)
```

Equivalent mathematical form:

```
Integer'First <= A - B <= Integer'Last
```

where $A - B$ is interpreted as unbounded mathematical integer subtraction, not machine `Integer` subtraction.

Memorise: Weakest error-free arithmetic precondition = "the mathematical result fits in the machine type."

Reason: This condition is exactly strong enough: if the mathematical result is inside the `Integer` range, the subtraction is safe; if it is outside the range, the Ada result cannot be represented. Converting operands to `Big_Integer` before subtracting avoids overflow inside the precondition itself.

Q11. SPARK Modular Proof Obligations - 4 marks

Consider the following SPARK Ada code:

```
procedure Bump_Or_Reset (I : in out Integer) with
```

```

    Pre => I >= -1,
    Post => I >= 0
is
begin
    if I < 0 then
        I := 0;
    else
        I := I + 1;
    end if;
end Bump_Or_Reset;

procedure Client (I : in out Integer) with
    Pre => I = 0,
    Post => I > 0
is
begin
    Bump_Or_Reset(I);
end Client;

```

Suppose the SPARK Prover is run on this code.

Describe the main checks it would perform and state whether each check should succeed or fail. Include:

- the call from Client to Bump_Or_Reset,
- the postcondition of Client,
- the postcondition of Bump_Or_Reset, and
- possible runtime errors inside Bump_Or_Reset.

Your answer:

Q11 Full-Mark Answer

The call from Client to Bump_Or_Reset should satisfy the callee precondition: Client has $\text{Pre} \Rightarrow I = 0$, and $0 \geq -1$, so this check succeeds.

The postcondition of Client is not provable modularly from the callee contract as written. After the call, SPARK may use only Bump_Or_Reset's postcondition, which gives $I \geq 0$; this is not strong enough to prove $I > 0$.

For Bump_Or_Reset's own postcondition, the functional reasoning is:

- If $I < 0$, the procedure sets $I := 0$, so $I \geq 0$.
- If $I \geq 0$, it sets $I := I + 1$, so the result would be positive if the addition is safe.

However, SPARK should report a possible runtime error in $I := I + 1$, because the precondition $I \geq -1$ still allows $I = \text{Integer}'\text{Last}$, and then $I + 1$ overflows.

Memorise: SPARK checks caller satisfies callee Pre, callee proves own Post, caller proves own Post from callee Post, and no runtime errors.

Reason: Modular proof does not automatically use the body of Bump_Or_Reset when proving Client; it uses the contract. Also, proving a postcondition is not enough if an arithmetic operation inside the body can overflow.

Q12. Hoare Logic: Loop Invariant - 4 marks

The following program counts how many of the first N elements of array A are positive. The valid indexes are 0 to $N - 1$, and you may assume $N \geq 0$.

```

count := 0;
i := 0;
while i != N do
    if A[i] > 0 then
        count := count + 1
    else
        skip
    endif;
    i := i + 1
done

```

Define:

$\text{positives}(k)$ = number of indexes j such that $0 \leq j \leq k - 1$ and $A[j] > 0$

The desired postcondition is:

$\text{count} = \text{positives}(N)$

Write a loop invariant strong enough to prove the postcondition.

$I =$

Q12 Full-Mark Answer

$I = 0 \leq i \leq N$ and $\text{count} = \text{positives}(i)$

Optional stronger version:

$I = 0 \leq i \leq N$ and $0 \leq \text{count} \leq i$ and $\text{count} = \text{positives}(i)$

Memorise: For array loops, invariant usually says: bounds on index + answer is correct for checked prefix.

Reason: At loop index i , the program has checked exactly the prefix $0 \dots i - 1$, not the whole array. Therefore count should equal $\text{positives}(i)$, not $\text{positives}(N)$, while the loop is still running.

Q13. Hoare Logic: Proof Sketch - 5 marks

Continuing the positive-counting program from Q12.

Using your invariant, give a proof sketch that the program establishes:

$\text{count} = \text{positives}(N)$

Your proof should cover:

1. initialization,
2. preservation for both branches of the `if`, and
3. loop exit.

Proof sketch:

Q13 Full-Mark Answer

Use the invariant:

$I = 0 \leq i \leq N$ and $\text{count} = \text{positives}(i)$

Initialization: After $\text{count} := 0$; $i := 0$, no array elements have been checked. The number of positives in the empty prefix is $\text{positives}(0) = 0$, so $\text{count} = \text{positives}(i)$ holds, and $0 \leq i \leq N$ holds because $N \geq 0$.

Preservation: Assume the invariant holds at the start of the loop and $i \neq N$. Then $i < N$, so $A[i]$ is a valid element.

- If $A[i] > 0$, the program increments count. The checked prefix grows from length i to length $i + 1$, and the new element is positive, so $\text{positives}(i + 1) = \text{positives}(i) + 1$. After $i := i + 1$, the invariant still holds.
- If $A[i] \leq 0$, count is unchanged. The checked prefix still grows to $i + 1$, but the new element is not positive, so $\text{positives}(i + 1) = \text{positives}(i)$. After $i := i + 1$, the invariant still holds.

Exit: When the loop exits, not $(i \neq N)$, so $i = N$. The invariant gives $\text{count} = \text{positives}(i)$, therefore $\text{count} = \text{positives}(N)$.

Memorise: Loop proof = init, preserve each branch, exit invariant + not guard $\rightarrow$ postcondition.

Reason: The invariant does not need to say the final result is already true during the loop. It says the partial result is correct for the prefix already checked.

Q14. Pointers and Separation Logic - 3 marks

Consider the following C program:

```
void copy_value(int *p, int *q) {  
    *p = *q;  
}
```

1. [1 mark] In ordinary pointer reasoning, why can aliasing between p and q make reasoning harder?
2. [2 marks] Write a separation logic precondition and postcondition for this program using points-to assertions and separating conjunction, assuming initially p points to value a and q points to value b, and p and q are distinct heap locations.

Aliasing explanation:

Precondition:

Postcondition:

Q14 Full-Mark Answer

Aliasing makes pointer reasoning harder because p and q might refer to the same heap cell. Then writing through p can also change what is observed through q, so the proof must consider alias and non-alias cases.

Precondition:

$p \mapsto a \ * \ q \mapsto b$

Postcondition:

$p \mapsto b \ * \ q \mapsto b$

The corresponding separation-logic triple is:

```
{ p ↦ a * q ↦ b }  
  *p = *q;  
{ p ↦ b * q ↦ b }
```

Memorise: * means separate heap cells. $p \mapsto a \ * \ q \mapsto b$ already says p and q are distinct.

Reason: The command reads value b from q and stores it into p. Because the precondition uses separating conjunction, the write to p cannot overwrite q's cell, so q still points to b.